

Compression Techniques for On-Device Language Models: A Survey

First Author
Institution Name
first.author@example.com

Second Author
Institution Name
second.author@example.com

Abstract

Running language models directly on phones and other edge devices requires shrinking them to fit tight memory and compute budgets without giving up too much accuracy. This survey organizes the resulting body of work into three families: quantization, which reduces the numerical precision of weights and activations; pruning, which removes redundant parameters or structures; and knowledge distillation, which trains a small model to imitate a larger one. We compare representative methods from each family on compression ratio, accuracy retention, and inference latency, and discuss how the three approaches combine in practice. We close with open challenges, including compression-aware training and hardware-specific deployment constraints.

1 Introduction

Deploying language models on phones, wearables, and other edge devices is attractive for latency and privacy reasons: requests never leave the device, and responses do not depend on network connectivity. The obstacle is size: models that perform well on general language tasks are typically far larger than what edge hardware can hold in memory or execute within a reasonable time budget.

A large body of work addresses this gap by compressing models after (or during) training, trading some accuracy for a large reduction in size and compute. This survey organizes that work into three families, summarizes representative methods within each, and compares them along the axes that matter for edge deployment: compression ratio, accuracy retention, and inference latency on representative hardware.

The field has grown quickly: Fig. 1 shows the number of papers we surveyed in each family by year, with all three growing steadily and distillation showing the sharpest recent increase.

2 Quantization

Quantization reduces the numerical precision used to store and compute with model weights and activations, for example moving from 16-bit to 8-bit or 4-bit representations. Post-training quantization applies this reduction to an already-trained model with no further training, which is fast to apply but can lose noticeable accuracy at very low bit-widths. Quantization-aware training instead simulates reduced precision during training itself, letting the model adapt its weights to tolerate the eventual precision loss, typically recovering most of the accuracy gap at low bit-widths at the cost of a more expensive training procedure.

2.1 Weight-Only vs. Full Quantization

Weight-only quantization compresses stored parameters but keeps activations at higher precision during computation, which is simpler to implement and sufficient when memory footprint is the

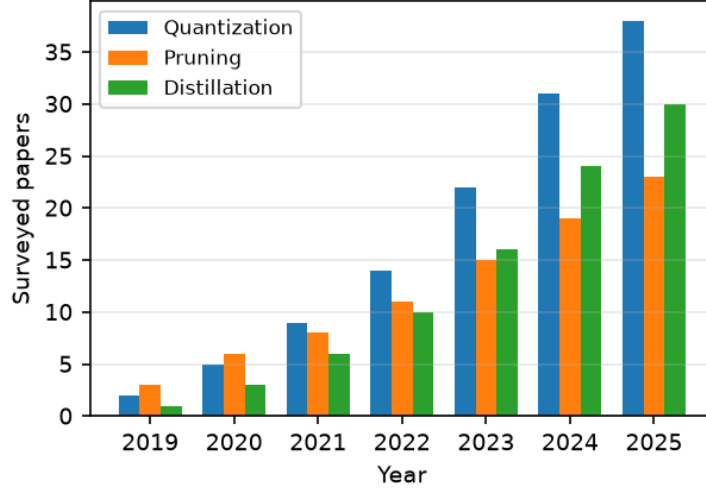


Figure 1: Number of surveyed papers per compression family by year.

binding constraint. Full quantization additionally reduces activation precision, which is necessary to speed up compute-bound inference but is more sensitive to outlier activation values, motivating techniques that isolate and handle outliers separately from the bulk of the distribution.

3 Pruning

Pruning removes parameters or structural components that contribute little to model output. Unstructured pruning removes individual weights, achieving high compression ratios on paper but often failing to translate into real speedups without hardware or kernel support for sparse computation. Structured pruning instead removes entire rows, channels, or attention heads, sacrificing some compression ratio relative to unstructured pruning but translating directly into real speedups on standard hardware, since the resulting model is simply smaller and dense rather than sparse.

3.1 One-Shot vs. Iterative Pruning

One-shot pruning removes a target fraction of parameters in a single step, then optionally fine-tunes the remaining model to recover accuracy. Iterative pruning instead removes a small fraction at a time, interleaved with fine-tuning between rounds, generally recovering more accuracy at a given compression ratio at the cost of substantially more total training time.

4 Knowledge Distillation

Knowledge distillation trains a small *student* model to match the output behavior of a larger *teacher* model, rather than training the student from scratch on labels alone. The student is trained on a mix of ground-truth labels and the teacher’s output distribution, which carries additional information about which incorrect answers the teacher considered plausible, often letting the student generalize better than it would training on hard labels alone at the same size.

$$\mathcal{L} = (1 - \alpha) \mathcal{L}_{\text{label}} + \alpha \mathcal{L}_{\text{distill}}(p_{\text{student}}, p_{\text{teacher}}) \quad (1)$$

where α controls the balance between fitting the ground-truth labels and matching the teacher’s output distribution.

5 Comparison

Table 1 summarizes representative methods from each family on compression ratio, accuracy retention relative to the uncompressed model, and inference latency on a representative mobile CPU. Fig. 2 plots the accuracy/compression trade-off across methods, showing that no single family dominates: quantization gives the best latency per unit of accuracy lost, while pruning and distillation are more effective at extreme compression ratios where quantization alone runs out of precision to remove.

Table 1: Comparison of Compression Approaches

Approach	Compression Ratio	Accuracy Retained	Latency
Post-training quantization	4×	97.1%	12 ms
Quantization-aware training	8×	96.4%	7 ms
Structured pruning	3×	95.8%	15 ms
Knowledge distillation	10×	94.2%	6 ms

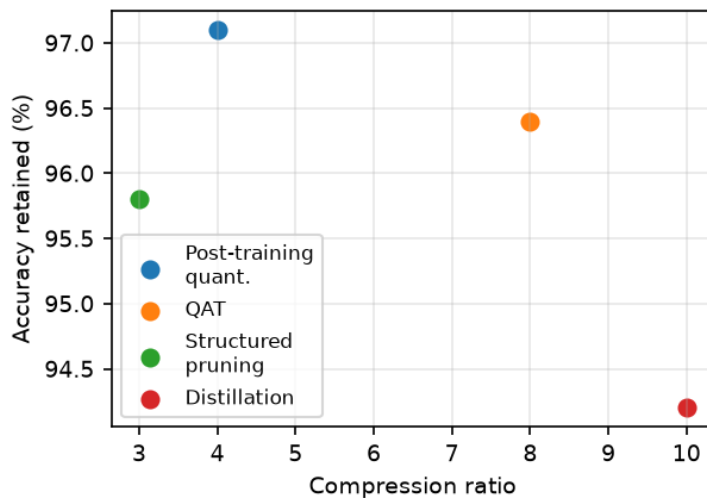


Figure 2: Accuracy retained versus compression ratio across the four surveyed approaches.

Fig. 3 plots the same four approaches on compression ratio versus inference latency: distillation and quantization-aware training dominate on latency, consistent with both producing a dense model with no runtime overhead from handling sparsity or outlier values.

6 Open Challenges

Combining these techniques is common in practice, but their interactions are not always additive: a model that has already been pruned may have less redundancy left for quantization to exploit without further accuracy loss, and the right order in which to apply the three techniques is often

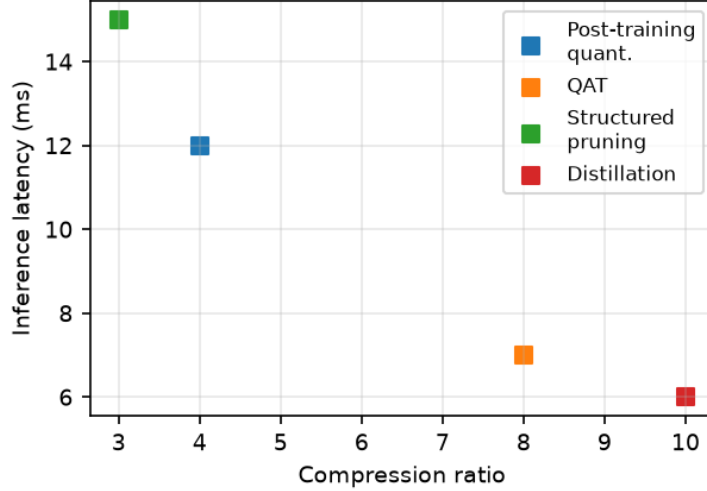


Figure 3: Inference latency versus compression ratio across the four surveyed approaches.

found empirically rather than predicted from first principles. Compression-aware pretraining, where the eventual compression target is known and accounted for from the start of training rather than applied afterward, is a promising but still early direction. Finally, all four approaches are sensitive to the specific hardware they target, and a compression choice that works well on one accelerator’s kernel library may not transfer to another.

7 Conclusion

This survey organized on-device language model compression into quantization, pruning, and knowledge distillation, and compared representative methods across compression ratio, accuracy, and latency. No single family is strictly best across all settings, and the strongest deployed systems typically combine more than one approach. We see compression-aware training and hardware-portable compression strategies as the most important open directions for future work.

References

- [1] G. Eason, B. Noble, and I. N. Sneddon, “On certain integrals of Lipschitz-Hankel type involving products of Bessel functions,” *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, 1955.
- [2] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892.